Explications du projet Sokoban (version pedagogique)

Ce document explique le projet comme une personne qui reprend un code deja ecrit par d'autres membres de l'equipe. Le but est de comprendre clairement : - ce que la prof demande dans Project 1 Part 1, - ou chaque exigence se trouve dans le code, - et la logique des fonctions importantes (BFS, successorFunction, isGoal).

---

1) Ce que la prof demande (resume simple)

La consigne Project 1 Part 1 est organisee en 3 parties.

1. A. Problem formulation

- Modeliser l'etat du jeu Sokoban.
- Modeliser le noeud de recherche.

2. B. Searching for a solution

- Resoudre avec BFS (Breadth-First Search), en suivant le pseudo-code du cours.

3. C. Game interface

- Faire une interface Pygame qui montre la simulation de la solution, son cout, et les informations de recherche.

Reference projet : - docs/Project 1 Part 1.pdf - docs/SPEC.md (copie de la specification utilisee dans le repo)

---

2) Ou se trouvent ces parties dans le code

A. Modeling the state

- sokoban.py
- Classe : SokobanPuzzle
- Fonctions importantes : **init**, isGoal, successorFunction

A. Modeling the node

- node.py
- Classe : Node
- Champs : state, parent, action, g
- Fonctions : getPath, getSolution

B. BFS

- search.py
- Fonction : BFS(s, successorsFn, isGoal)

C. Game interface

- gui.py
- Classe : Game
- Lancement de BFS dans la GUI : methode resoudre
- Affichage/animation dans la boucle update + draw

Point d'entree

- main.py
- python main.py (GUI)
- python main.py –console (test BFS en console)

---

3) Modelisation de l'etat (A.1) - logique detaillee

Fichier : sokoban.py

3.1 Comment un etat est represente

Un etat Sokoban est : - une grille de caracteres, - plus la position du joueur.

Symboles utilises dans la grille : - R : joueur - O : mur - espace : case vide - S : cible - B : caisse - . : joueur sur cible - * : caisse sur cible

3.2 Pourquoi le code utilise tuple(grid)

La grille est stockee en tuple de chaines : - c'est immuable, - c'est hashable, - donc on peut l'utiliser dans des sets pour BFS.

C'est important pour reconnaitre rapidement un etat deja vu.

3.3 Egalite et hash

- **eq** compare les grilles.
- **hash** calcule hash(self.grid).

Donc deux etats avec la meme grille sont consideres identiques, meme s'ils sont des objets Python differents.

---

4) successorFunction - explication tres simple

Fichier : sokoban.py

4.1 Role

successorFunction repond a la question : "Depuis cet etat, quels coups sont legalement possibles ?"

La reponse est une liste de paires : (action, nouvel_etat)

4.2 Ce que la fonction fait pour chaque direction

Le code essaie toujours : UP, DOWN, LEFT, RIGHT.

Pour chaque direction : 1. regarder la case devant le joueur (n1) 2. selon le contenu de n1 : - si mur O : coup interdit - si vide ou cible : deplacement simple - si caisse B ou * : - regarder la case derriere la caisse (n2) - si n2 est libre (vide ou cible), pousser autorise - sinon pousser interdit

4.3 Ce que la fonction garantit

Cette fonction applique les regles officielles Sokoban : - pas de passage dans les murs, - pas de tirage de caisse, - pas de poussee de deux caisses, - une poussee seulement si la case derriere est libre.

4.4 Tres important

L'etat parent n'est jamais modifie. Le code construit une nouvelle grille pour chaque successeur.

---

5) isGoal - explication claire

Fichier : sokoban.py

isGoal retourne vrai si la grille ne contient plus aucun B.

Pourquoi c'est correct : - B = caisse pas encore sur cible, - * = caisse deja sur cible.

Donc "plus de B" veut dire "toutes les caisses sont sur cibles".

---

6) Modelisation du noeud (A.2)

Fichier : node.py

Un Node contient exactement ce qui est demande : - state : etat Sokoban - parent : noeud parent - action : action qui a cree ce noeud - g : cout du chemin

Ici chaque pas coute 1, donc : - g de la racine = 0 - sinon g = parent.g + 1

Fonctions : - getPath : reconstruit la suite d'etats depuis l'initial - getSolution : reconstruit la suite d'actions

---

7) BFS (partie B) - explication pas a pas

Fichier : search.py

7.1 Idee intuitive

BFS explore par couches : - d'abord distance 0, - puis distance 1, - puis distance 2, etc.

Avec un cout de pas constant (=1), le premier but trouve est optimal.

7.2 Structures

- ouvert : file FIFO (deque)
- ferme : liste des noeuds deja developpes

- ouvertSet et fermeSet : optimisation de performance

7.3 Deroulement concret

1. creer la racine Node(etat_initial)
2. si racine est deja but -> retour immediat
3. mettre racine dans Open
4. tant que Open non vide :

- sortir le premier noeud
- le mettre dans Closed
- generer ses successeurs
- pour chaque successeur non deja vu :
  - si but : retour
  - sinon ajouter dans Open

5. si Open vide sans but : pas de solution

7.4 Difference entre cout et steps

Dans ce code, BFS renvoie : (noeud_but_ou_None, steps)

- cout g : longueur de la solution finale
- steps : nombre de noeuds developpes par BFS

steps mesure le travail de recherche, pas la longueur de la solution.

---

8) Interface de jeu (partie C)

Fichier : gui.py

8.1 Ce que fait l'interface

- menu de niveaux
- jeu manuel (fleches)
- bouton Resoudre (BFS)
- affichage du resultat BFS
- lecture automatique de la solution
- animations de victoire

8.2 Point architecture important

La GUI ne recode pas les regles de Sokoban. Elle appelle successorFunction et applique seulement les coups legalement proposes.

Donc les regles restent centralisees dans sokoban.py.

8.3 Cycle principal

La classe Game tourne avec : - handle_events - update - draw

Cette separation rend le code plus lisible : - entree utilisateur, - logique etat, - affichage.

---

9) Verification de conformite a la consigne

Base de comparaison : docs/Project 1 Part 1.pdf (et docs/SPEC.md du repo).

A. Modeling state

Exigences : symboles, mouvements, init/isGoal/successorFunction. Etat : conforme.

A. Modeling node

Exigences : state, parent, action, g, getPath, getSolution. Etat : conforme.

B. BFS

Exigences : BFS selon pseudo-code du cours. Etat : conforme (avec optimisation sets qui ne change pas la logique).

C. Interface Pygame

Exigences : simuler la solution et afficher les infos de recherche. Etat : conforme.

Conclusion : Le code respecte les points centraux demandes par la prof pour A, B, C.

---

10) Lecture conseillee si tu presentes a l'oral

Ordre simple : 1. main.py (point d'entree) 2. sokoban.py (regles) 3. node.py (structure de noeud) 4. search.py (BFS) 5. gui.py (integration visuelle) 6. test_core.py (preuves de bon fonctionnement)

Si tu bloques pendant l'oral, tu peux dire : - "Les regles du jeu sont centralisees dans successorFunction." - "BFS travaille avec Open/Closed, evite les etats deja vus, et reconstruit la solution via parent." - "isGoal est vrai quand il ne reste plus de B."

---

11) Commandes utiles

- Lancer le jeu :
  - python main.py
- Lancer BFS console :
  - python main.py –console
- Lancer les tests :
  - python test_core.py